

Chapter 5

Polynomials

*The polynomial $(z - 1)(z - 2) \dots (z - 20)$ is not a 'difficult' polynomial per se ...
The 'difficulty' with the polynomial $\prod (z - i)$ is that of
evaluating the explicit polynomial accurately.
If one already knows the roots, then the polynomial can be evaluated
without any loss of accuracy.*

— J. H. WILKINSON, *The Perfidious Polynomial* (1984)

*I first used backward error analysis in connection with
simple programs for computing zeros of polynomials
soon after the PILOT ACE came into use.*

— J. H. WILKINSON, *The State of the Art in Error Analysis* (1985)

*The Fundamental Theorem of Algebra asserts that
every polynomial equation over the complex field has a root.
It is almost beneath the dignity of such a majestic theorem
to mention that in fact it has precisely n roots.*

— J. H. WILKINSON, *The Perfidious Polynomial* (1984)

*It can happen ... that a particular polynomial
can be evaluated accurately by nested multiplication,
whereas evaluating the same polynomial by an economical method
may produce relatively inaccurate results.*

— C. T. FIKE, *Computer Evaluation of Mathematical Functions* (1968)

Two common tasks associated with polynomials are evaluation and interpolation: given the polynomial find its values at certain arguments, and given the values at certain arguments find the polynomial. We consider Horner's rule for evaluation and the Newton divided difference polynomial for interpolation. A third task not considered here is finding the zeros of a polynomial. Much research was devoted to polynomial zero finding up until the late 1960s; indeed, Wilkinson devotes a quarter of *Rounding Errors in Algebraic Processes* [1232, 1963] to the topic. Since the development of the QR algorithm for finding matrix eigenvalues there has been less demand for polynomial zero finding, since the problem either arises as, or can be converted to (see §28.6 and [383, 1995], [1144, 1994]), the matrix eigenvalue problem.

5.1. Horner's Method

The standard method for evaluating a polynomial

$$p(x) = a_0 + a_1x + \cdots + a_nx^n \quad (5.1)$$

is Horner's method (also known as Horner's rule and nested multiplication), which consists of the following recurrence:

$$\begin{aligned} q_n(x) &= a_n \\ \text{for } i &= n-1: -1: 0 \\ q_i(x) &= xq_{i+1}(x) + a_i \\ \text{end} \\ p(x) &= q_0(x) \end{aligned}$$

The cost is $2n$ flops, which is n less than the more obvious method of evaluation that explicitly forms powers of x (see Problem 5.2).

To analyse the rounding errors in Horner's method it is convenient to use the relative error counter notation $\langle k \rangle$ (see (3.10)). We have

$$\begin{aligned} \hat{q}_{n-1} &= (x\hat{q}_n\langle 1 \rangle + a_{n-1})\langle 1 \rangle \\ &= xa_n\langle 2 \rangle + a_{n-1}\langle 1 \rangle, \\ \hat{q}_{n-2} &= (x\hat{q}_{n-1}\langle 1 \rangle + a_{n-2})\langle 1 \rangle \\ &= x^2a_n\langle 4 \rangle + xa_{n-1}\langle 3 \rangle + a_{n-2}\langle 1 \rangle. \end{aligned}$$

It is easy to either guess or prove by induction that

$$\begin{aligned} \hat{q}_0 &= a_0\langle 1 \rangle + a_1x\langle 3 \rangle + \cdots + a_{n-1}x^{n-1}\langle 2n-1 \rangle + a_nx^n\langle 2n \rangle \\ &= (1 + \theta_1)a_0 + (1 + \theta_3)a_1x + \cdots + (1 + \theta_{2n-1})a_{n-1}x^{n-1} \\ &\quad + (1 + \theta_{2n})a_nx^n, \end{aligned} \quad (5.2)$$

where we have used Lemma 3.1, and where $|\theta_k| \leq ku/(1 - ku) =: \gamma_k$. This result shows that Horner's method has a small backward error: the computed $\hat{q}_0$ is the exact value at x of a polynomial obtained by making relative perturbations of size at most γ_{2n} to the coefficients of $p(x)$.

A forward error bound is easily obtained: from (5.2) we have

$$|p(x) - \hat{q}_0| \leq \gamma_{2n} \sum_{i=0}^n |a_i| |x|^i = \gamma_{2n} \tilde{p}(|x|), \quad (5.3)$$

where $\tilde{p}(x) = \sum_{i=0}^n |a_i| x^i$. The relative error is bounded according to

$$\frac{|p(x) - \hat{q}_0|}{|p(x)|} \leq \gamma_{2n} \frac{\tilde{p}(|x|)}{|p(x)|} =: \gamma_{2n} \psi(p, x).$$

Clearly, the factor $\psi(p, x)$ can be arbitrarily large. However, $\psi(p, x) = 1$ if $a_i \geq 0$ for all i and $x \geq 0$, or if $(-1)^i a_i \geq 0$ for all i and $x \leq 0$.

In a practical computation we may wish to compute an error bound along with $\hat{q}_0$. The bound (5.3) is entirely adequate for theoretical purposes and can itself be computed by Horner's method. However, it lacks sharpness for two reasons. First, the bound is the result of replacing each γ_k by γ_{2n} . Second, and more importantly, it is an a priori bound and so takes no account of the actual rounding errors that occur. We can derive a sharper, a posteriori bound by a running error analysis.

For the i th step of Horner's method we can write

$$(1 + \epsilon_i) \hat{q}_i = x \hat{q}_{i+1} (1 + \delta_i) + a_i, \quad |\delta_i|, |\epsilon_i| \leq u, \quad (5.4)$$

where we have used both (2.4) and (2.5). Defining $\hat{q}_i =: q_i + f_i$, we have

$$q_i + f_i + \epsilon_i \hat{q}_i = x(q_{i+1} + f_{i+1}) + x \hat{q}_{i+1} \delta_i + a_i,$$

or

$$f_i = x f_{i+1} + x \hat{q}_{i+1} \delta_i - \epsilon_i \hat{q}_i, \quad f_n = 0.$$

Hence

$$|f_i| \leq |x| |f_{i+1}| + u(|x| |\hat{q}_{i+1}| + |\hat{q}_i|).$$

Since $f_n = 0$, we have $|f_i| \leq u \pi_i$, where

$$\pi_i = |x| \pi_{i+1} + |x| |\hat{q}_{i+1}| + |\hat{q}_i|, \quad \pi_n = 0.$$

We can slightly reduce the cost of evaluating the majorizing sequence π_i by working with $\mu_i := \frac{1}{2}(\pi_i + |\hat{q}_i|)$, which satisfies the recurrence

$$\mu_i = |x| \mu_{i+1} + |\hat{q}_i|, \quad \mu_n = \frac{1}{2} |\hat{q}_n|.$$

We can now furnish Horner's method with a running error bound.

Algorithm 5.1. This algorithm evaluates $y = fl(p(x))$ by Horner's method, where $p(x) = \sum_{i=0}^n a_i x^i$. It also evaluates a quantity μ such that $|y - p(x)| \leq \mu$.

```

y = a_n
μ = |y|/2
for i = n - 1 : -1 : 0
    y = xy + a_i
    μ = |x|μ + |y|
end
μ = u(2μ - |y|)

```

Cost: $4n$ flops.

It is worth commenting on the case where one or more of the a_i and x is complex. The analysis leading to Algorithm 5.1 is still valid for complex data, but we need to remember that the error bounds for $fl(xopy)$ are not the same as for real arithmetic. In view of Lemma 3.5, it suffices to replace the last line of the algorithm by $\mu = \sqrt{2}\gamma_2(2\mu - |y|)$. An increase in speed of the algorithm, with only a slight worsening of the bound, can be obtained by replacing $|y| = ((\operatorname{Re} y)^2 + (\operatorname{Im} y)^2)^{1/2}$ by $|\operatorname{Re} y| + |\operatorname{Im} y|$ (and, of course, $|x|$ should be evaluated once and for all before entering the loop).

One use of Algorithm 5.1 is to provide a stopping criterion for a polynomial zero-finder: if $|fl(p(x))|$ is of the same order as the error bound μ , then further iteration serves no purpose, for as far as we can tell, x could be an exact zero.

As a numerical example, for the expanded form of $p(x) = (x + 1)^{32}$ we found in MATLAB that

$$fl(p(-1)) \equiv 0, \quad \mu = 2.4 \times 10^{-7}, \quad \gamma_{2n}\tilde{p}(|x|) = 1.5 \times 10^{-5},$$

and for $p(x)$ the Chebyshev polynomial of degree 32,

$$fl(p(0.5)) \approx 0.5000, \quad \mu = 3.3 \times 10^{-10}, \quad \gamma_{2n}\tilde{p}(|x|) = 1.0 \times 10^{-8}.$$

In these two cases, the running error bound is, respectively, 62 and 31 times smaller than the a priori one.

In another experiment we evaluated the expanded form of $p(x) = (x - 2)^3$ in simulated single precision in MATLAB ($u \approx 6 \times 10^{-8}$) for 200 equally spaced points near $x = 2$. The polynomial values, the error, and the a priori and running error bounds are all plotted in Figure 5.1. The running error bound is about seven times smaller than the a priori one.

5.2. Evaluating Derivatives

Suppose now that we wish to evaluate derivatives of p . We could simply differentiate (5.1) as many times as necessary and apply Horner's method to each expression, but there is a more efficient way. Observe that if we define

$$q(x) = q_1 + q_2x + \cdots + q_nx^{n-1}, \quad r = q_0,$$

where the $q_i = q_i(\alpha)$ are generated by Horner's method for $p(\alpha)$, then

$$p(x) = (x - \alpha)q(x) + r.$$

In other words, Horner's method carries out the process of synthetic division. Clearly, $p'(\alpha) = q(\alpha)$. It is worth noting in passing that for $x \neq \alpha$,

$$q(x) = \frac{p(x) - p(\alpha)}{x - \alpha},$$

that is, q is a divided difference. If we repeat synthetic division recursively on $q(x)$, we will be evaluating the coefficients in the Taylor expansion

$$p(x) = p(\alpha) + (x - \alpha)p'(\alpha) + \frac{(x - \alpha)^2}{2!}p''(\alpha) + \cdots + \frac{(x - \alpha)^n}{n!}p^{(n)}(\alpha),$$

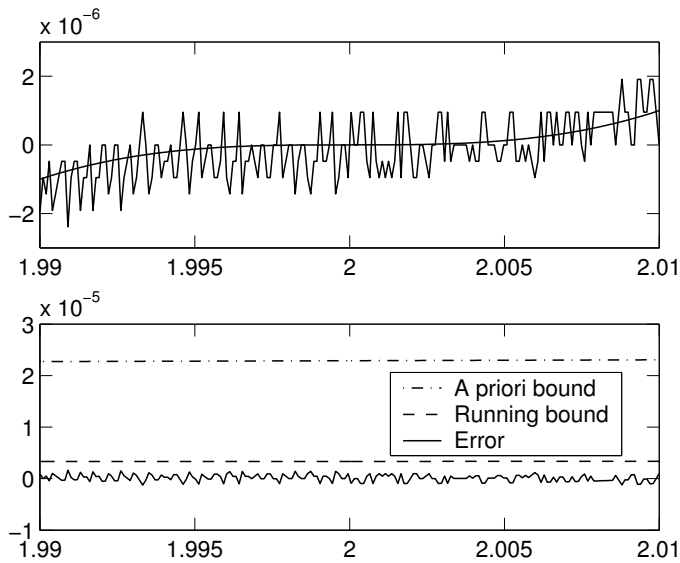


Figure 5.1. *Computed polynomial values (top) and running and a priori bounds (bottom) for Horner's method.*

and after a final scaling by factorials, we will obtain the derivatives of p at α . The resulting algorithm is quite short.

Algorithm 5.2. This algorithm evaluates the polynomial $p(x) = \sum_{i=0}^n a_i x^i$ and its first k derivatives at α , returning $y_i = p^{(i)}(\alpha)$, $i = 0:k$.

```

 $y_0 = a_n$ 
 $y(1:k) = 0$ 
for  $j = n - 1 : -1 : 0$ 
    for  $i = \min(k, n - j) : -1 : 1$ 
         $y_i = \alpha y_i + y_{i-1}$ 
    end
     $y_0 = \alpha y_0 + a_j$ 
end
 $m = 1$ 
for  $j = 2 : k$ 
     $m = m * j$ 
     $y_j = m * y_j$ 
end

```

Cost: $nk + 2(k + n) - k^2/2$ flops.

How is the error bounded for the derivatives in Algorithm 5.2? To answer this question with the minimum of algebra, we express the algorithm in matrix notation. Horner's method for evaluating $p(\alpha)$ is equivalent to solution of the

bidiagonal system

$$U_{n+1}q := \begin{bmatrix} 1 & -\alpha & & & \\ & 1 & -\alpha & & \\ & & 1 & \ddots & \\ & & & \ddots & -\alpha \\ & & & & 1 \end{bmatrix} \begin{bmatrix} q_0 \\ q_1 \\ \vdots \\ \vdots \\ q_n \end{bmatrix} = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ \vdots \\ a_n \end{bmatrix} =: a.$$

By considering (5.4), we see that

$$(U_{n+1} + \Delta_1)\hat{q} = a, \quad |\Delta_1| \leq u|U_{n+1}|.$$

Hence

$$|q - \hat{q}| \leq u|U_{n+1}^{-1}||U_{n+1}||q| + O(u^2). \quad (5.5)$$

The recurrence for $r_0 = p'(\alpha)$ can be expressed as $U_n r = q(1:n)$, where $r = r(0:n-1)$, so

$$(U_n + \Delta_2)\hat{r} = \hat{q}(1:n), \quad |\Delta_2| \leq u|U_n|.$$

Hence

$$\begin{aligned} \hat{r} &= (U_n^{-1} - U_n^{-1}\Delta_2 U_n^{-1})(q(1:n) + (\hat{q}(1:n) - q(1:n))) + O(u^2) \\ &= r - U_n^{-1}\Delta_2 r + U_n^{-1}(\hat{q}(1:n) - q(1:n)) + O(u^2). \end{aligned}$$

This gives, using (5.5),

$$|r - \hat{r}| \leq u|U_n^{-1}||U_n||r| + u|U_n^{-1}||U_n^{-1}||U_n||q(1:n)| + O(u^2). \quad (5.6)$$

Now

$$\begin{aligned} |U_n^{-1}| &= \begin{bmatrix} 1 & |\alpha| & |\alpha|^2 & \dots & |\alpha|^{n-1} \\ & 1 & |\alpha| & & \vdots \\ & & 1 & \ddots & \vdots \\ & & & \ddots & |\alpha| \\ & & & & 1 \end{bmatrix}, \\ |U_n^{-1}||U_n| &= \begin{bmatrix} 1 & 2|\alpha| & 2|\alpha|^2 & \dots & 2|\alpha|^{n-1} \\ & 1 & 2|\alpha| & & \vdots \\ & & 1 & \ddots & \vdots \\ & & & \ddots & 2|\alpha| \\ & & & & 1 \end{bmatrix}, \\ |U_n^{-1}||U_n^{-1}||U_n| &= \begin{bmatrix} 1 & 3|\alpha| & 5|\alpha|^2 & \dots & (2n-1)|\alpha|^{n-1} \\ & 1 & 3|\alpha| & & \vdots \\ & & 1 & \ddots & \vdots \\ & & & \ddots & 3|\alpha| \\ & & & & 1 \end{bmatrix}. \end{aligned}$$

By looking at the form of r and q , we find from (5.6) that

$$\begin{aligned}
 |p'(\alpha) - \widehat{r}_0| &\leq 2u \sum_{k=1}^n k^2 |a_k| |\alpha|^{k-1} + O(u^2) \\
 &\leq 2nu \sum_{k=1}^n k |a_k| |\alpha|^{k-1} + O(u^2) \\
 &=: 2nu \widehat{p}'(\alpha) + O(u^2).
 \end{aligned} \tag{5.7}$$

This is essentially the same form of bound as for $p(\alpha)$ in (5.3). Analogous bounds hold for all derivatives.

5.3. The Newton Form and Polynomial Interpolation

An alternative to the monomial representation of a polynomial is the Newton form

$$p(x) = \sum_{i=0}^n c_i \prod_{j=0}^{i-1} (x - \alpha_j), \tag{5.8}$$

which is commonly used for polynomial interpolation. The interpolation problem is to choose p so that $p(\alpha_i) = f_i$, $i = 0:n$, and the numbers c_i are known as divided differences. Assuming that the points α_j are distinct, the divided differences may be computed from a standard recurrence:

```

 $c^{(0)}(0:n) = f(0:n)$ 
for  $k = 0:n-1$ 
  for  $j = n-1:k+1$ 
     $c_j^{(k+1)} = (c_j^{(k)} - c_{j-1}^{(k)}) / (\alpha_j - \alpha_{j-k-1})$ 
  end
end
 $c = c^{(n)}$ 

```

Cost: $3n^2/2$ flops.

Two questions are of interest: how accurate are the computed $\widehat{c}_j$, and what is the effect of rounding errors on the polynomial values obtained by evaluating the Newton form? To answer the first question we express the recurrence in matrix-vector form:

$$c^{(0)} = f, \quad c^{(k+1)} = L_k c^{(k)}, \quad k = 0:n-1,$$

where $L_k = D_k^{-1} M_k$ is lower bidiagonal, with

$$D_k = \text{diag}(\text{ones}(1:k+1), \alpha_{k+1} - \alpha_0, \alpha_{k+2} - \alpha_1, \dots, \alpha_n - \alpha_{n-k-1}),$$

$$M_k = \begin{bmatrix} I_k & & & & \\ & 1 & & & \\ & -1 & 1 & & \\ & & -1 & \ddots & \\ & & & \ddots & \ddots \\ & & & & -1 & 1 \end{bmatrix}.$$

It is straightforward to show that

$$\widehat{c}^{(k+1)} = G_k L_k \widehat{c}^{(k)}, \quad (5.9)$$

where $G_k = \text{diag}(\text{ones}(1:k+1), \eta_{k,k+2}, \dots, \eta_{k,n+1})$, where each η_{ij} is of the form $\eta_{ij} = (1 + \delta_1)(1 + \delta_2)(1 + \delta_3)$, $|\delta_i| \leq u$. Hence

$$\widehat{c} = (L_{n-1} + \Delta L_{n-1}) \dots (L_0 + \Delta L_0) f, \quad |\Delta L_i| \leq \gamma_3 |L_i|. \quad (5.10)$$

From Lemma 3.8,

$$\begin{aligned} |c - \widehat{c}| &= |(L_{n-1} \dots L_0 - (L_{n-1} + \Delta L_{n-1}) \dots (L_0 + \Delta L_0)) f| \\ &\leq ((1 + \gamma_3)^n - 1) |L_{n-1}| \dots |L_0| |f| \\ &= ((1 - 3u)^{-n} - 1) |L_{n-1}| \dots |L_0| |f|. \end{aligned} \quad (5.11)$$

To interpret the bound, note first that merely rounding the data ($f_i \rightarrow f_i(1 + \delta_i)$, $|\delta_i| \leq u$) can cause an error Δc as large as $e_{\text{round}} = u|L||f|$, where $L = L_{n-1} \dots L_0$, so errors of at least this size are inevitable. Since $|L_{n-1}| \dots |L_0| \geq |L_{n-1} \dots L_0| = |L|$, the error in the computed divided differences can be larger than e_{round} only if there is much subtractive cancellation in the product $L = L_{n-1} \dots L_0$. If $\alpha_0 < \alpha_1 < \dots < \alpha_n$ then each L_i is positive on the diagonal and nonpositive on the first subdiagonal; therefore $|L_{n-1}| \dots |L_0| = |L_{n-1} \dots L_0| = |L|$, and we have the very satisfactory bound $|c - \widehat{c}| \leq ((1 - 3u)^{-n} - 1)|L||f|$. This same bound holds if the α_i are arranged in decreasing order.

To examine how well the computed Newton form reproduces the f_i we “unwind” the analysis above. From (5.9) we have

$$\widehat{c}^{(k)} = L_k^{-1} G_k^{-1} \widehat{c}^{(k+1)} = (L_k^{-1} + \widetilde{\Delta L_k}) \widehat{c}^{(k+1)}, \quad |\widetilde{\Delta L_k}| \leq \gamma_3 |L_k^{-1}|.$$

By invoking Lemma 3.8 again, we obtain

$$|f - L^{-1} \widehat{c}| \leq ((1 - 3u)^{-n} - 1) |L_0^{-1}| \dots |L_{n-1}^{-1}| |\widehat{c}|. \quad (5.12)$$

If $\alpha_0 < \alpha_1 < \dots < \alpha_n$ then $L_i^{-1} \geq 0$ for all i , and we obtain the very satisfactory bound $|f - L^{-1} \widehat{c}| \leq ((1 - 3u)^{-n} - 1) |L^{-1}| |\widehat{c}|$. Again, the same bound holds for points arranged in decreasing order.

In practice it is found that even when the computed divided differences are very inaccurate, the computed interpolating polynomial may still reproduce the original data well. The bounds (5.11) and (5.12) provide insight into this observed behaviour by showing that $c - \widehat{c}$ and $f - L^{-1} \widehat{c}$ can be large only when there is much cancellation in the products $L_{n-1} \dots L_0 f$ and $L_0^{-1} \dots L_{n-1}^{-1} c$, respectively.

The analysis has shown that the ordering $\alpha_0 < \alpha_1 < \dots < \alpha_n$ yields “optimal” error bounds for the divided differences and the residual, and so may be a good choice of ordering of interpolation points. However, if the aim is to minimize $|p(x) - fl(p(x))|$ for a given $x \neq \alpha_j$, then other orderings need to be considered. An ordering with some theoretical support is the Leja ordering, which is defined by the equations [975, 1990]

$$\alpha_0 = \max_i |\alpha_i|, \quad (5.13a)$$

$$\prod_{k=0}^{j-1} |\alpha_j - \alpha_k| = \max_{i \geq j} \prod_{k=0}^{j-1} |\alpha_i - \alpha_k|, \quad j = 1:n-1. \quad (5.13b)$$

For a given set of $n + 1$ points α_i , the Leja ordering can be computed in n^2 flops (see Problem 5.4).

We give a numerical example to illustrate the analysis. Let $n = 16$ and let $\alpha_0 < \dots < \alpha_n$ be equally spaced points on $[-1, 1]$. Working in simulated single precision with $u = 2^{-24} \approx 6 \times 10^{-8}$, we computed divided differences for two different vectors f . Error statistics were computed by regarding the solutions computed in double precision as exact. We define the ratios

$$\rho_1 = \max_i \frac{(|L_{n-1}| \dots |L_0| |f|)_i}{|c_i|}, \quad \rho_2 = \max_i \frac{(|L_0^{-1}| \dots |L_{n-1}^{-1}| |\hat{c}|)_i}{|f_i|}.$$

(1) For f_i from the normal $N(0, 1)$ distribution the divided differences range in magnitude from 1 to 10^5 , and their relative errors range from 0 (the first divided difference, f_0 , is always exact) to 3×10^{-7} . The ratio $\rho_1 = 16.3$, so (5.11) provides a reasonably sharp bound for the error in $\hat{c}$. The relative errors when f is reconstructed from the computed divided differences range between 0 and 3×10^{-1} (it makes little difference whether the reconstruction is done in single or double precision). Again, this is predicted by the analysis, in this case by (5.12), because $\rho_2 = 2 \times 10^7$. For the Leja ordering, the divided differences are computed with about the same accuracy, but f is reconstructed much more accurately, with maximum relative error 7×10^{-6} ($\rho_1 = 1 \times 10^3$, $\rho_2 = 8 \times 10^4$).

(2) For $f_i = \exp(\alpha_i)$, the situation is reversed: we obtain inaccurate divided differences but an accurate reconstruction of f . The divided differences range in magnitude from 10^{-4} to 10^{-1} , and their relative errors are as large as 1, but the relative errors in the reconstructed f are all less than 10^{-7} . Again, the error bounds predict this behaviour: $\rho_1 = 6 \times 10^8$, $\rho_2 = 1.02$. The Leja ordering performs similarly.

The natural way to evaluate the polynomial (5.8) for a given x is by a generalization of Horner's method:

```

 $q_n(x) = c_n$ 
for  $i = n - 1 : -1 : 0$ 
     $q_i(x) = (x - \alpha_i)q_{i+1}(x) + c_i$ 
end
 $p(x) = q_0(x)$ 

```

A straightforward analysis shows that (cf. (5.2))

$$\begin{aligned} \hat{q}_0 &= c_0 \langle 1 \rangle + (x - \alpha_0) c_1 \langle 4 \rangle + (x - \alpha_0)(x - \alpha_1) c_2 \langle 7 \rangle + \dots \\ &\quad + (x - \alpha_0) \dots (x - \alpha_{n-2}) c_{n-1} \langle 3n - 2 \rangle \\ &\quad + (x - \alpha_0) \dots (x - \alpha_{n-1}) c_n \langle 3n \rangle. \end{aligned}$$

Hence the computed $\hat{q}_0$ is the exact value corresponding to a polynomial with slightly perturbed divided differences. The corresponding forward error bound is

$$|p(x) - \hat{q}_0| \leq \gamma_{3n} \sum_{i=0}^n |c_i| \prod_{j=0}^{i-1} |x - \alpha_j|.$$

5.4. Matrix Polynomials

The scalar polynomial (5.1) can be generalized to a matrix polynomial in three ways: to a matrix polynomial with scalar coefficients and matrix argument,

$$P_1(x) = a_0I + a_1X + \cdots + a_nX^n, \quad a_i \in \mathbb{C}, \quad X \in \mathbb{C}^{m \times m};$$

to a matrix polynomial with matrix coefficients and scalar argument,

$$P_2(x) = A_0 + A_1x + \cdots + A_nx^n, \quad A_i \in \mathbb{C}^{m \times m}, \quad x \in \mathbb{C};$$

and to a matrix polynomial with matrix coefficients and matrix argument,

$$P_3(x) = A_0 + A_1X + \cdots + A_nX^n, \quad A_i \in \mathbb{C}^{m \times m}, \quad X \in \mathbb{C}^{m \times m}. \quad (5.14)$$

(We could also consider an analogue of P_3 in which the coefficient matrices A_i appear to the right of the powers of X .) Each type of polynomial arises in applications: P_1 in the approximation of matrix functions such as the exponential and logarithm [868, 1978], [509, 1996, Chap. 11], [615, 2001]; P_2 in the polynomial eigenvalue problem [1139, 2001]; and P_3 in quasi-birth-death processes in Markov chains and in numerical solution of the polynomial eigenvalue problem [617, 2000].

Evaluation of P_2 (or, more commonly, P_2y for some vector y) is straightforward. To evaluate P_1 and P_3 Horner's method can be used, but for P_1 Horner's method is not necessarily of optimal efficiency, even for a single evaluation. In fact, a method of Paterson and Stockmeyer can evaluate P_1 in a number of matrix multiplications proportional to $\sqrt{n}$, at the cost of extra storage proportional to $m^2\sqrt{n}$ elements [928, 1973], [509, 1996, §11.2.4]. The storage can be reduced to a small constant multiple of n^2 by a modification of Van Loan, which slightly increases the cost [1179, 1979].

Error analysis of Horner's method and of the Paterson–Stockmeyer method for evaluating P_1 and P_3 is straightforward; see Problem 5.6.

5.5. Notes and References

Backward and forward error analysis for Horner's rule was given by Wilkinson [1232, 1963, pp. 36–37, 49–50]; our results are simply Wilkinson's presented in a different notation. The analysis has been redone by many other authors, sometimes without reference to Wilkinson's results. Another early reference, which gives a forward error bound only, is McCracken and Dorn [833, 1964, §3.5],

For more on running error analysis see §3.3.

Müller [877, 1983] gives a first-order error analysis for the evaluation of the divided difference form of a polynomial. Olver [905, 1986] derives a posteriori error bounds for the Horner scheme with derivatives (Algorithm 5.2), phrasing them in terms of his relative precision notation. Stewart [1063, 1971] analyses synthetic division, using a matrix-oriented approach similar to that in §5.2.

The relative merits of the monomial and Chebyshev representations of a polynomial are investigated, with respect to accuracy of evaluation, by Newbery [888, 1974] and Schonfelder and Razaz [1020, 1980]. Clenshaw [239, 1955] showed how Horner's method could be extended to evaluate a polynomial expressed in the

Chebyshev form $p(x) = \sum_{i=0}^n a_i T_i(x)$, where T_i is the Chebyshev polynomial of degree i . Error analysis of Clenshaw's method, and variations of it, are given by Gentleman [472, 1969], Newbery [887, 1973], and Oliver [901, 1977], [902, 1979]. Clenshaw's scheme can be generalized to expansions in terms of arbitrary orthogonal polynomials; see Smith [1050, 1965] and Algorithm 22.8.

Running error bounds for Horner's method were included in algorithms of Kahan and Farkas [703, 1963], [704, 1963] without explanation. Adams [6, 1967] derives the bounds and extends them to evaluation of a real polynomial at a complex argument. Algorithm 5.1 is given in [6, 1967], and also in the classic paper by Peters and Wilkinson [937, 1971], which describes many aspects of the solution of polynomial equations. Wilkinson's paper "The Perfidious Polynomial" [1247, 1984] (for which he was awarded the Chauvenet Prize) is highly recommended as a beautifully written introduction to backward error analysis in general and error analysis for polynomials in particular.

There seems to be little work on choosing the ordering of interpolation points to minimize the effect of rounding errors on the construction or evaluation of the interpolating polynomial. Werner [1216, 1984] examines experimentally the effect of different orderings on the computed value of an interpolating polynomial at a single point, for several forms of interpolating polynomial.

The Leja ordering, which was proposed by Leja in a 1957 paper, is analysed in detail by Reichel [975, 1990]. He shows that an appropriately defined condition number for the Newton form of interpolating polynomial grows at a slower than exponential rate in the degree n for Leja points, which are points taken from a given compact set that satisfy the condition (5.13). For more on the numerical benefits of the Leja ordering see §22.3.3.

Eğecioglu, Gallopoulos, and Koç [396, 1990] give a parallel algorithm, based on the parallel prefix operation, for computing divided differences and evaluating the interpolating polynomial and they give a rounding error analysis of the algorithm.

If a polynomial is to be evaluated many times at different arguments it may be worthwhile to expend some effort transforming it to a form that can be evaluated more cheaply than by a straightforward application of Horner's rule. For example, the quartic

$$p(x) = a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0, \quad a_4 \neq 0,$$

can be rewritten as [744, 1998, Sec. 4.6.4]

$$p(x) = ((y + x + \alpha_2)y + \alpha_3)\alpha_4, \quad y = (x + \alpha_0)x + \alpha_1,$$

where the coefficients α_i are given by

$$\begin{aligned} \alpha_0 &= \frac{1}{2}(a_3/a_4 - 1), & \beta &= a_2/a_4 - \alpha_0(\alpha_0 + 1), & \alpha_1 &= a_1/a_4 - \alpha_0\beta, \\ \alpha_2 &= \beta - 2\alpha_1, & \alpha_3 &= a_0/a_4 - \alpha_1(\alpha_1 + \alpha_2), & \alpha_4 &= a_4. \end{aligned}$$

Once the α_i have been computed, $p(x)$ can be evaluated in three multiplications and five additions, as compared with the four multiplications and four additions required by Horner's rule. If a multiplication takes longer than an addition, the transformed polynomial should be cheaper to evaluate. For polynomials of degree

$n > 4$ there exist evaluation schemes that require strictly less than the $2n$ total additions and multiplications required by Horner's rule; see Knuth [741, 1962], [744, 1998, Sec. 4.6.4] and Fike [412, 1967]. One application in which such schemes have been used is in evaluating polynomial approximations in an elementary function library [451, 1991]. Little seems to be known about the numerical stability of fast polynomial evaluation schemes; see Problem 5.7.

Problems

5.1. Give an alternative derivation of Algorithm 5.2 by differentiating the Horner recurrence and rescaling the iterates.

5.2. Give an error analysis for the following “beginner’s” algorithm for evaluating $p(x) = a_0 + a_1x + \cdots + a_nx^n$:

```

 $q(x) = a_0; y = 1$ 
for  $i = 1:n$ 
     $y = xy$ 
     $q(x) = q(x) + a_iy$ 
end
 $p(x) = q(x)$ 

```

5.3. Let $p(x) = a_0 + a_1x + \cdots + a_nx^n$ and $n = 2m$. Then

$$\begin{aligned} p(x) &= (a_0 + a_2x^2 + \cdots + a_{2m}x^{2m}) + (a_1x + a_3x^3 + \cdots + a_{2m-1}x^{2m-1}) \\ &= a_0 + a_2y + \cdots + a_{2m}y^m + x(a_1 + a_3y + \cdots + a_{2m-1}y^{m-1}), \end{aligned}$$

where $y = x^2$. Obtain an error bound for $fl(p(x))$ when p is evaluated using this splitting (using Horner's rule on each part).

5.4. Write down an algorithm for computing the Leja ordering (5.13) in n^2 flops.

5.5. If the polynomial $p(x) = \sum_{i=0}^n a_i x^i$ has roots $x_1, \dots, x_n$, it can be evaluated from the root product form $p(x) = a_n \prod_{i=1}^n (x - x_i)$. Give an error analysis for this evaluation.

5.6. Show that the computed polynomial $\hat{P}_3$ from Horner's method applied to P_3 in (5.14) satisfies

$$\|P_3 - \hat{P}_3\| \leq n(m+1)u\tilde{p}_3(\|X\|) + O(u^2),$$

where $\tilde{p}_3(x) = \sum_{k=0}^n \|A_k\| x^k$ and the norm is the 1-norm or the ∞ -norm.

5.7. (RESEARCH PROBLEM) Investigate the numerical stability of fast polynomial evaluation schemes (see the Notes and References) by both rounding error analysis and numerical experiments. For a brief empirical study see Miller [851, 1975, §10].